

ACM India Summer School on Symmetric Key Cryptography

Nitin Kumar Sharma

Centre for Cryptography and Cybersecurity
IIT Hyderabad

June 15, 2026

Cryptography and Machine Learning

Insights from Ronald L. Rivest

Introduction: Cryptography Versus Machine Learning

- Cryptography and Machine Learning (ML) are deeply intertwined and have been since the 1940s and 50s.

Introduction: Cryptography Versus Machine Learning

- Cryptography and Machine Learning (ML) are deeply intertwined and have been since the 1940s and 50s.
- **Cryptanalysis as Learning:** Breaking a cipher can be viewed as the problem of "**learning an unknown function**" (the decryption function) from examples of its input/output behavior.

Mapping Terminology

Cryptography	Machine Learning

Mapping Terminology

Cryptography	Machine Learning
Attacker	

Mapping Terminology

Cryptography	Machine Learning
Attacker	Learner

Mapping Terminology

Cryptography	Machine Learning
Attacker	Learner
Secret Key	

Mapping Terminology

Cryptography	Machine Learning
Attacker	Learner
Secret Key	Target Function

Mapping Terminology

Cryptography	Machine Learning
Attacker	Learner
Secret Key	Target Function
Key Space	

Mapping Terminology

Cryptography	Machine Learning
Attacker	Learner
Secret Key	Target Function
Key Space	Class of Possible Target Functions

Mapping Terminology

Cryptography	Machine Learning
Attacker	Learner
Secret Key	Target Function
Key Space	Class of Possible Target Functions
Plaintext	Input Data (X)

Mapping Terminology

Cryptography	Machine Learning
Attacker	Learner
Secret Key	Target Function
Key Space	Class of Possible Target Functions
Plaintext	Input Data (X)
Ciphertext	Output (Y)

Mapping Terminology

Cryptography	Machine Learning
Attacker	Learner
Secret Key	Target Function
Key Space	Class of Possible Target Functions
Plaintext	Input Data (X)
Ciphertext	Output (Y)
Cryptanalysis	

Mapping Terminology

Cryptography	Machine Learning
Attacker	Learner
Secret Key	Target Function
Key Space	Class of Possible Target Functions
Plaintext	Input Data (X)
Ciphertext	Output (Y)
Cryptanalysis	Learning Algorithm

Mapping Terminology

Cryptography	Machine Learning
Attacker	Learner
Secret Key	Target Function
Key Space	Class of Possible Target Functions
Plaintext	Input Data (X)
Ciphertext	Output (Y)
Cryptanalysis	Learning Algorithm
Total Break/Secret key	

Mapping Terminology

Cryptography	Machine Learning
Attacker	Learner
Secret Key	Target Function
Key Space	Class of Possible Target Functions
Plaintext	Input Data (X)
Ciphertext	Output (Y)
Cryptanalysis	Learning Algorithm
Total Break/Secret key	Exact Identification

Mapping Terminology

Cryptography	Machine Learning
Attacker	Learner
Secret Key	Target Function
Key Space	Class of Possible Target Functions
Plaintext	Input Data (X)
Ciphertext	Output (Y)
Cryptanalysis	Learning Algorithm
Total Break/Secret key	Exact Identification

Table: Correspondences between the two fields.

How Information is Gathered

Both fields specify protocols for how an attacker (or learner) interacts with the unknown system:

How Information is Gathered

Both fields specify protocols for how an attacker (or learner) interacts with the unknown system:

Cryptographic Attacks

- Ciphertext-only

How Information is Gathered

Both fields specify protocols for how an attacker (or learner) interacts with the unknown system:

Cryptographic Attacks

- Ciphertext-only
- Known Plaintext (KPA)

How Information is Gathered

Both fields specify protocols for how an attacker (or learner) interacts with the unknown system:

Cryptographic Attacks

- Ciphertext-only
- Known Plaintext (KPA)
- Chosen Plaintext (CPA)

How Information is Gathered

Both fields specify protocols for how an attacker (or learner) interacts with the unknown system:

Cryptographic Attacks

- Ciphertext-only
- Known Plaintext (KPA)
- Chosen Plaintext (CPA)
- Chosen Ciphertext (CCA)

ML Learning Protocols

- Membership queries -asking for the value of the unknown function on some specified input.

ML Learning Protocols

- Membership queries -asking for the value of the unknown function on some specified input.
- Random examples are drawn according to an arbitrary but fixed probability distribution that is unknown to the learner.

ML Learning Protocols

- Membership queries -asking for the value of the unknown function on some specified input.
- Random examples are drawn according to an arbitrary but fixed probability distribution that is unknown to the learner.
- Equivalence Queries (Testing a hypothesis).

Key Distinctions

Despite similarities, the fields differ in their goals:

Distribution Independence

Machine learning requires learning for *all* underlying distributions, while **Cryptographic Security** must hold even if the message distribution is fixed or known.

Key Distinctions

Despite similarities, the fields differ in their goals:

Distribution Independence

Machine learning requires learning for *all* underlying distributions, while **Cryptographic Security** must hold even if the message distribution is fixed or known.

Exact vs. Approximate

Cryptanalysts usually want a "Total Break" (exact key). **ML Learners** often settle for "Approximate Extraction" (a function that is 99% correct).

Summary

- Cryptography provides the "shield" (hardness results) that tests the limits of Learning Theory.

Summary

- Cryptography provides the "shield" (hardness results) that tests the limits of Learning Theory.
- Machine Learning provides the "sword" (new algorithms) that can potentially find patterns in cryptographic data.

Summary

- Cryptography provides the "shield" (hardness results) that tests the limits of Learning Theory.
- Machine Learning provides the "sword" (new algorithms) that can potentially find patterns in cryptographic data.
- Understanding the terminology mapping allows researchers in one field to benefit from the breakthroughs in the other.

Summary

- Cryptography provides the "shield" (hardness results) that tests the limits of Learning Theory.
- Machine Learning provides the "sword" (new algorithms) that can potentially find patterns in cryptographic data.
- Understanding the terminology mapping allows researchers in one field to benefit from the breakthroughs in the other.
- **Future Goal:** Learning theory may help cryptographers build more effective compression algorithms to hide data patterns better

Motivation

- Machine learning today is one of the most important and fastest-growing fields of technology.

Motivation

- Machine learning today is one of the most important and fastest-growing fields of technology.
- Applications of machine learning are becoming omnipresent, and solutions learned from data are increasingly displacing traditional hand-crafted algorithms.

Motivation

- Machine learning today is one of the most important and fastest-growing fields of technology.
- Applications of machine learning are becoming omnipresent, and solutions learned from data are increasingly displacing traditional hand-crafted algorithms.
- This has not only led to improved performance for existing technologies but has opened the door to a vast range of new capabilities that would be inconceivable if new algorithms had to be designed explicitly by hand.

Motivation

- Machine learning today is one of the most important and fastest-growing fields of technology.
- Applications of machine learning are becoming omnipresent, and solutions learned from data are increasingly displacing traditional hand-crafted algorithms.
- This has not only led to improved performance for existing technologies but has opened the door to a vast range of new capabilities that would be inconceivable if new algorithms had to be designed explicitly by hand.
- One particular branch of machine learning, known as deep learning, has emerged as an exceptionally powerful and general-purpose framework for learning from data.

Pioneer Work

- Deep learning is based on computational models called neural networks, which were originally inspired by mechanisms of learning and information processing in the human brain.

Pioneer Work

- Deep learning is based on computational models called neural networks, which were originally inspired by mechanisms of learning and information processing in the human brain.
- A new cryptanalysis technique emerged based on deep learning: neural cryptanalysis. The possibility of using neural networks to perform cryptographic tasks has been studied since the 1990s [Riv91].

Pioneer Work

- Deep learning is based on computational models called neural networks, which were originally inspired by mechanisms of learning and information processing in the human brain.
- A new cryptanalysis technique emerged based on deep learning: neural cryptanalysis. The possibility of using neural networks to perform cryptographic tasks has been studied since the 1990s [Riv91].
- The first work to successfully use the power of neural networks to improve the state-of-the-art results on a particular cipher was proposed by Aron Gohr at CRYPTO 2019 [Goh19].

Pioneer Work

- Deep learning is based on computational models called neural networks, which were originally inspired by mechanisms of learning and information processing in the human brain.
- A new cryptanalysis technique emerged based on deep learning: neural cryptanalysis. The possibility of using neural networks to perform cryptographic tasks has been studied since the 1990s [Riv91].
- The first work to successfully use the power of neural networks to improve the state-of-the-art results on a particular cipher was proposed by Aron Gohr at CRYPTO 2019 [Goh19].
- Gohr's work exploits the ability of deep learning algorithms to recognize complex patterns to identify relations between sets of ciphertext that distinguish them from random data.

Pioneer Work

- In his work [Goh19], this relation is differential in nature; given pairs of ciphertexts

$$C_0 = E_K(M_0), C_1 = E_K(M_1),$$

where $E_K(X)$ denotes the encryption of X with the key K through a number of rounds of a block cipher. The neural distinguisher is trained to determine whether $M_0 \oplus M_1 = \Delta M$?

Pioneer Work

- In his work [Goh19], this relation is differential in nature; given pairs of ciphertexts

$$C_0 = E_K(M_0), C_1 = E_K(M_1),$$

where $E_K(X)$ denotes the encryption of X with the key K through a number of rounds of a block cipher. The neural distinguisher is trained to determine whether $M_0 \oplus M_1 = \Delta M$?

- Gohr's neural distinguishers on 5, 6, 7, and 8 rounds of SPECK32 enabled key recovery attacks for 11 and 12 rounds with better complexity than the state of the art.

Pioneer Work

- In his work [Goh19], this relation is differential in nature; given pairs of ciphertexts

$$C_0 = E_K(M_0), C_1 = E_K(M_1),$$

where $E_K(X)$ denotes the encryption of X with the key K through a number of rounds of a block cipher. The neural distinguisher is trained to determine whether $M_0 \oplus M_1 = \Delta M$?

- Gohr's neural distinguishers on 5, 6, 7, and 8 rounds of SPECK32 enabled key recovery attacks for 11 and 12 rounds with better complexity than the state of the art.
- As this field is very young, it is not clear what the true potential of neural cryptanalysis is, and which limitations it will encounter in the future. Moreover, there are unexplored paths related to machine learning, but not directly to neural networks, to perform cryptanalysis of symmetric ciphers, which can possibly lead to new research directions.

Distinguisher Types

Purely Differential Distinguishers

Exploit only the observed bitwise differences between ciphertext pairs.

Distinguisher Types

Purely Differential Distinguishers

Exploit only the observed bitwise differences between ciphertext pairs.

General Differential Distinguishers (Neural)

Exploit the full distribution of ciphertext pairs, including features "invisible" to purely differential tools.

Distinguisher Types

Purely Differential Distinguishers

Exploit only the observed bitwise differences between ciphertext pairs.

General Differential Distinguishers (Neural)

Exploit the full distribution of ciphertext pairs, including features "invisible" to purely differential tools.

- Neural networks can distinguish pairs even when their difference distributions are identical.

Classical vs. Neural: Two Types of Detectives

The Classical Detective (Purely Differential)

Looks only at the **difference** between the two scrambled messages. It's like a detective looking only for fingerprints.

Classical vs. Neural: Two Types of Detectives

The Classical Detective (Purely Differential)

Looks only at the **difference** between the two scrambled messages. It's like a detective looking only for fingerprints.

Neural Network

Looks at **everything**—the bits, the patterns, and the subtle relationships humans can't even describe. It's like a detective looking for fingerprints, the smell of the room, and the temperature of the air.

Beyond Human Cryptanalysis

- **Few-Shot Learning:** The model can solve cryptographic tasks after seeing as few as 1 to 50 examples.

Beyond Human Cryptanalysis

- **Few-Shot Learning:** The model can solve cryptographic tasks after seeing as few as 1 to 50 examples.
- **Automated Discovery:** Neural networks can find optimal input differences without human guidance.

Beyond Human Cryptanalysis

- **Few-Shot Learning:** The model can solve cryptographic tasks after seeing as few as 1 to 50 examples.
- **Automated Discovery:** Neural networks can find optimal input differences without human guidance.
- **Bayesian Optimization:** A highly efficient, sequential strategy used to find the maximum or minimum of complex, "black-box" functions. When combined with neural tools, it reduces the attack complexity.

Conclusions

- Neural networks are a powerful, **complementary tool** for modern cryptanalysis.

Conclusions

- Neural networks are a powerful, **complementary tool** for modern cryptanalysis.
- They excel at finding non-linear features that humans might miss.

Conclusions

- Neural networks are a powerful, **complementary tool** for modern cryptanalysis.
- They excel at finding non-linear features that humans might miss.
- While they won't replace traditional methods, they significantly enhance the "cryptographer's toolbox".

The Attacker's Goal: The Distinguishing Game

- To break a cipher, you first have to tell if a message is actually encrypted or just random noise.

The Attacker's Goal: The Distinguishing Game

- To break a cipher, you first have to tell if a message is actually encrypted or just random noise.
- **The Task:** An attacker looks at a pair of scrambled messages. They must guess:

The Attacker's Goal: The Distinguishing Game

- To break a cipher, you first have to tell if a message is actually encrypted or just random noise.
- **The Task:** An attacker looks at a pair of scrambled messages. They must guess:
 - 1 Is this a "Real" encrypted pair?

The Attacker's Goal: The Distinguishing Game

- To break a cipher, you first have to tell if a message is actually encrypted or just random noise.
- **The Task:** An attacker looks at a pair of scrambled messages. They must guess:
 - 1 Is this a "Real" encrypted pair?
 - 2 Or is this just "Random" junk?

The Attacker's Goal: The Distinguishing Game

- To break a cipher, you first have to tell if a message is actually encrypted or just random noise.
- **The Task:** An attacker looks at a pair of scrambled messages. They must guess:
 - 1 Is this a "Real" encrypted pair?
 - 2 Or is this just "Random" junk?
- If you can guess better than 50/50, you've found a "leak" in the cipher's security.

Improving Attacks on Round-Reduced Speck32/64 Using Deep Learning

Based on the paper by Aron Gohr(CRYPTO'19)

Short Summary I

- Gohr use machine learning to build differential distinguishers **superior to the full difference distribution table** for round-reduced Speck.

Short Summary I

- Gohr use machine learning to build differential distinguishers **superior to the full difference distribution table** for round-reduced Speck.
- Use of *Bayesian optimization* to obtain a key search policy that efficiently uses their distinguishers. This leads to an attack that uses very few trial decryptions (**millions of times less** than previous state of the art).

Short Summary I

- Gohr use machine learning to build differential distinguishers **superior to the full difference distribution table** for round-reduced Speck.
- Use of *Bayesian optimization* to obtain a key search policy that efficiently uses their distinguishers. This leads to an attack that uses very few trial decryptions (**millions of times less** than previous state of the art).
- The resulting attack on Speck32/64 reduced to 11 rounds is $\approx$ **200 times faster** than previous records.

Short Summary II

- Main attack does **not use the key schedule**, so in particular does not need the key schedule to be efficiently invertible.

Short Summary II

- Main attack does **not use the key schedule**, so in particular does not need the key schedule to be efficiently invertible.
- Neural distinguishers can efficiently recognize $C_0 \oplus T, C_1 \oplus T$ as not a real ciphertext pair (even when $C_0 \oplus C_1$ is a **high-probability output difference**).

Short Summary II

- Main attack does **not use the key schedule**, so in particular does not need the key schedule to be efficiently invertible.
- Neural distinguishers can efficiently recognize $C_0 \oplus T, C_1 \oplus T$ as not a real ciphertext pair (even when $C_0 \oplus C_1$ is a **high-probability output difference**).
- They make use of few-shot learning to **find good input differences** without using prior cryptanalysis.

The Target: Speck32/64

Speck Family:

- Designed by the NSA (Beaulieu et al.) in 2013.

The Target: Speck32/64

Speck Family:

- Designed by the NSA (Beaulieu et al.) in 2013.
- Optimized for software performance on IoT devices.

The Target: Speck32/64

Speck Family:

- Designed by the NSA (Beaulieu et al.) in 2013.
- Optimized for software performance on IoT devices.
- **ARX Construction:** Addition (mod 2^{16}), Rotation, XOR.

The Target: Speck32/64

Speck Family:

- Designed by the NSA (Beaulieu et al.) in 2013.
- Optimized for software performance on IoT devices.
- **ARX Construction:** Addition (mod 2^{16}), Rotation, XOR.

The Target: Speck32/64

Speck Family:

- Designed by the NSA (Beaulieu et al.) in 2013.
- Optimized for software performance on IoT devices.
- **ARX Construction:** Addition (mod 2^{16}), Rotation, XOR.

Round Function (F):

- State: Two 16-bit words (L_i, R_i).

The Target: Speck32/64

Speck Family:

- Designed by the NSA (Beaulieu et al.) in 2013.
- Optimized for software performance on IoT devices.
- **ARX Construction:** Addition (mod 2^{16}), Rotation, XOR.

Round Function (F):

- State: Two 16-bit words (L_i, R_i) .
- Update rule:

$$L_{i+1} = ((L_i \ggg \alpha) + R_i) \oplus K$$

$$R_{i+1} = (R_i \lll \beta) \oplus L_{i+1}$$

The Target: Speck32/64

Speck Family:

- Designed by the NSA (Beaulieu et al.) in 2013.
- Optimized for software performance on IoT devices.
- **ARX Construction:** Addition (mod 2^{16}), Rotation, XOR.

Round Function (F):

- State: Two 16-bit words (L_i, R_i) .
- Update rule:

$$L_{i+1} = ((L_i \ggg \alpha) + R_i) \oplus K$$

$$R_{i+1} = (R_i \lll \beta) \oplus L_{i+1}$$

The Target: Speck32/64

Speck Family:

- Designed by the NSA (Beaulieu et al.) in 2013.
- Optimized for software performance on IoT devices.
- **ARX Construction:** Addition (mod 2^{16}), Rotation, XOR.

Round Function (F):

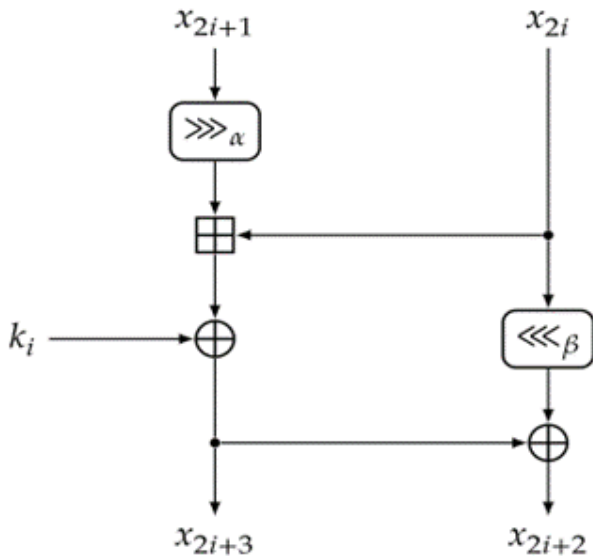
- State: Two 16-bit words (L_i, R_i) .
- Update rule:

$$L_{i+1} = ((L_i \ggg \alpha) + R_i) \oplus K$$

$$R_{i+1} = (R_i \lll \beta) \oplus L_{i+1}$$

- Constants for Speck32/64: $\alpha = 7, \beta = 2$.

Speck Design



Classical Differential Cryptanalysis

The Standard:

- Analyzes the propagation of input differences Δ_{in} to output differences Δ_{out} .

Markov Assumption:

Classical Differential Cryptanalysis

The Standard:

- Analyzes the propagation of input differences Δ_{in} to output differences Δ_{out} .
- Probability defined as:

$$P(\Delta_{in} \rightarrow \Delta_{out}) = \frac{|\{x : F(x) \oplus F(x \oplus \Delta_{in}) = \Delta_{out}\}|}{2^n}$$

Markov Assumption:

Classical Differential Cryptanalysis

The Standard:

- Analyzes the propagation of input differences Δ_{in} to output differences Δ_{out} .
- Probability defined as:

$$P(\Delta_{in} \rightarrow \Delta_{out}) = \frac{|\{x : F(x) \oplus F(x \oplus \Delta_{in}) = \Delta_{out}\}|}{2^n}$$

Markov Assumption:

- Assumes differential transition probabilities are independent of plaintext values (if subkeys are random).

Classical Differential Cryptanalysis

The Standard:

- Analyzes the propagation of input differences Δ_{in} to output differences Δ_{out} .
- Probability defined as:

$$P(\Delta_{in} \rightarrow \Delta_{out}) = \frac{|\{x : F(x) \oplus F(x \oplus \Delta_{in}) = \Delta_{out}\}|}{2^n}$$

Markov Assumption:

- Assumes differential transition probabilities are independent of plaintext values (if subkeys are random).
- Used to calculate the full **Difference Distribution Table (DDT)**.

Classical Differential Cryptanalysis

The Standard:

- Analyzes the propagation of input differences Δ_{in} to output differences Δ_{out} .
- Probability defined as:

$$P(\Delta_{in} \rightarrow \Delta_{out}) = \frac{|\{x : F(x) \oplus F(x \oplus \Delta_{in}) = \Delta_{out}\}|}{2^n}$$

Markov Assumption:

- Assumes differential transition probabilities are independent of plaintext values (if subkeys are random).
- Used to calculate the full **Difference Distribution Table (DDT)**.
- Baseline:** The paper calculates the full DDT for up to 8 rounds to serve as the "perfect" classical baseline.

The Distinguishing Task

Objective: Distinguish real Speck output from random data.

The Distinguishing Task

Objective: Distinguish real Speck output from random data.

- **Input Difference (Δ):** Fixed at 0x0040/0000.

Baseline Distinguisher:

The Distinguishing Task

Objective: Distinguish real Speck output from random data.

- **Input Difference (Δ):** Fixed at 0x0040/0000.
- **Real Pair:** (C_0, C_1) where $C_0 = E_k(P)$ and $C_1 = E_k(P \oplus \Delta)$.

Baseline Distinguisher:

The Distinguishing Task

Objective: Distinguish real Speck output from random data.

- **Input Difference (Δ):** Fixed at $0x0040/0000$.
- **Real Pair:** (C_0, C_1) where $C_0 = E_k(P)$ and $C_1 = E_k(P \oplus \Delta)$.
- **Random (Blinded) Pair:** $(\tilde{C}_0, \tilde{C}_1)$ where $\tilde{C}_0 = C_0 \oplus K$, $\tilde{C}_1 = C_1 \oplus K$, and K is a uniformly random value from $\{0, 1\}^b$, where b is the block size of the primitive.

Baseline Distinguisher:

The Distinguishing Task

Objective: Distinguish real Speck output from random data.

- **Input Difference (Δ):** Fixed at $0x0040/0000$.
- **Real Pair:** (C_0, C_1) where $C_0 = E_k(P)$ and $C_1 = E_k(P \oplus \Delta)$.
- **Random (Blinded) Pair:** $(\tilde{C}_0, \tilde{C}_1)$ where $\tilde{C}_0 = C_0 \oplus K$, $\tilde{C}_1 = C_1 \oplus K$, and K is a uniformly random value from $\{0, 1\}^b$, where b is the block size of the primitive.

Baseline Distinguisher:

- Classify as *Real* if $P(\Delta_{out} | \text{Real}) > P(\Delta_{out} | \text{Random})$.

The Distinguishing Task

Objective: Distinguish real Speck output from random data.

- **Input Difference (Δ):** Fixed at $0x0040/0000$.
- **Real Pair:** (C_0, C_1) where $C_0 = E_k(P)$ and $C_1 = E_k(P \oplus \Delta)$.
- **Random (Blinded) Pair:** $(\tilde{C}_0, \tilde{C}_1)$ where $\tilde{C}_0 = C_0 \oplus K$, $\tilde{C}_1 = C_1 \oplus K$, and K is a uniformly random value from $\{0, 1\}^b$, where b is the block size of the primitive.

Baseline Distinguisher:

- Classify as *Real* if $P(\Delta_{out} | \text{Real}) > P(\Delta_{out} | \text{Random})$.
- Uses the precomputed DDT.

Limitations of Classical Analysis

Pure Differential Distinguishers:

- Only observe the difference $\Delta_{out} = C_0 \oplus C_1$.

The Challenge

Can a Neural Network learn to distinguish better than the theoretical limit of the DDT by looking at the raw ciphertext values?

Limitations of Classical Analysis

Pure Differential Distinguishers:

- Only observe the difference $\Delta_{out} = C_0 \oplus C_1$.
- Discard the actual ciphertext values (C_0, C_1) .

The Challenge

Can a Neural Network learn to distinguish better than the theoretical limit of the DDT by looking at the raw ciphertext values?

Limitations of Classical Analysis

Pure Differential Distinguishers:

- Only observe the difference $\Delta_{out} = C_0 \oplus C_1$.
- Discard the actual ciphertext values (C_0, C_1) .
- **Hypothesis:** Neural networks can exploit the full ciphertext distribution, not just the difference.

The Challenge

Can a Neural Network learn to distinguish better than the theoretical limit of the DDT by looking at the raw ciphertext values?

Network Structure: Input Representation

To apply Deep Learning to block ciphers, the input format is critical.

Network Structure: Input Representation

To apply Deep Learning to block ciphers, the input format is critical.

Input Data:

- A ciphertext pair (C_0, C_1).

Network Structure: Input Representation

To apply Deep Learning to block ciphers, the input format is critical.

Input Data:

- A ciphertext pair (C_0, C_1) .
- Represented as four 16-bit words: (w_0, w_1, w_2, w_3) .

Network Structure: Input Representation

To apply Deep Learning to block ciphers, the input format is critical.

Input Data:

- A ciphertext pair (C_0, C_1) .
- Represented as four 16-bit words: (w_0, w_1, w_2, w_3) .

Network Structure: Input Representation

To apply Deep Learning to block ciphers, the input format is critical.

Input Data:

- A ciphertext pair (C_0, C_1) .
- Represented as four 16-bit words: (w_0, w_1, w_2, w_3) .

Format:

- The words are interpreted as rows in a matrix.

Network Structure: Input Representation

To apply Deep Learning to block ciphers, the input format is critical.

Input Data:

- A ciphertext pair (C_0, C_1) .
- Represented as four 16-bit words: (w_0, w_1, w_2, w_3) .

Format:

- The words are interpreted as rows in a matrix.
- Input Layer: 4×16 array (64 units).

Network Structure: Input Representation

To apply Deep Learning to block ciphers, the input format is critical.

Input Data:

- A ciphertext pair (C_0, C_1) .
- Represented as four 16-bit words: (w_0, w_1, w_2, w_3) .

Format:

- The words are interpreted as rows in a matrix.
- Input Layer: 4×16 array (64 units).
- Preserves the word-oriented structure of Speck.

Network Architecture: ResNet

Structure: Deep Residual Neural Network (ResNet).

① Initial Convolution:

- Bit-sliced convolution (width 1).

② Residual Tower:

③ Prediction Head:

Network Architecture: ResNet

Structure: Deep Residual Neural Network (ResNet).

① Initial Convolution:

- Bit-sliced convolution (width 1).
- 32 output channels.

② Residual Tower:

③ Prediction Head:

Network Architecture: ResNet

Structure: Deep Residual Neural Network (ResNet).

① Initial Convolution:

- Bit-sliced convolution (width 1).
- 32 output channels.
- Purpose: Learn simple bitwise functions (like XOR).

② Residual Tower:

③ Prediction Head:

Network Architecture: ResNet

Structure: Deep Residual Neural Network (ResNet).

① Initial Convolution:

- Bit-sliced convolution (width 1).
- 32 output channels.
- Purpose: Learn simple bitwise functions (like XOR).

② Residual Tower:

- Depth: 1 to 10 residual blocks (depending on round count).

③ Prediction Head:

Network Architecture: ResNet

Structure: Deep Residual Neural Network (ResNet).

① Initial Convolution:

- Bit-sliced convolution (width 1).
- 32 output channels.
- Purpose: Learn simple bitwise functions (like XOR).

② Residual Tower:

- Depth: 1 to 10 residual blocks (depending on round count).
- Block: 2 layers of 32 filters + BatchNorm + ReLU + Skip Connection.

③ Prediction Head:

Network Architecture: ResNet

Structure: Deep Residual Neural Network (ResNet).

① Initial Convolution:

- Bit-sliced convolution (width 1).
- 32 output channels.
- Purpose: Learn simple bitwise functions (like XOR).

② Residual Tower:

- Depth: 1 to 10 residual blocks (depending on round count).
- Block: 2 layers of 32 filters + BatchNorm + ReLU + Skip Connection.

③ Prediction Head:

- Fully connected layers (Dense 64 $\rightarrow$ Dense 64).

Network Architecture: ResNet

Structure: Deep Residual Neural Network (ResNet).

① Initial Convolution:

- Bit-sliced convolution (width 1).
- 32 output channels.
- Purpose: Learn simple bitwise functions (like XOR).

② Residual Tower:

- Depth: 1 to 10 residual blocks (depending on round count).
- Block: 2 layers of 32 filters + BatchNorm + ReLU + Skip Connection.

③ Prediction Head:

- Fully connected layers (Dense 64 $\rightarrow$ Dense 64).
- Output: Single unit with Sigmoid activation.

Training a Distinguisher for Reduced Speck

- Generate a few million real and random samples with the input difference $\Delta = (0 \times 40, 0 \times 0)$ (this takes a few seconds on any reasonable computer).

Training a Distinguisher for Reduced Speck

- Generate a few million real and random samples with the input difference $\Delta = (0 \times 40, 0 \times 0)$ (this takes a few seconds on any reasonable computer).
- Train a *deep residual neural network* to distinguish the real from the random samples.

Training a Distinguisher for Reduced Speck

- Generate a few million real and random samples with the input difference $\Delta = (0 \times 40, 0 \times 0)$ (this takes a few seconds on any reasonable computer).
- Train a *deep residual neural network* to distinguish the real from the random samples.
- For 5 to 7 rounds of Speck encryption, better classifier than DDT after a few minutes of training on GPU (GTX 1080 Ti).

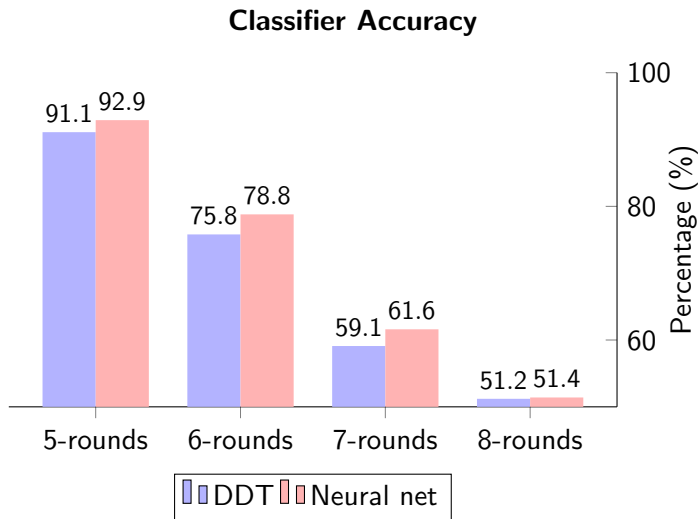
Training a Distinguisher for Reduced Speck

- Generate a few million real and random samples with the input difference $\Delta = (0 \times 40, 0 \times 0)$ (this takes a few seconds on any reasonable computer).
- Train a *deep residual neural network* to distinguish the real from the random samples.
- For 5 to 7 rounds of Speck encryption, better classifier than DDT after a few minutes of training on GPU (GTX 1080 Ti).
- For seven rounds, a more expensive training scheme gives significant further improvement to network performance.

Training a Distinguisher for Reduced Speck

- Generate a few million real and random samples with the input difference $\Delta = (0 \times 40, 0 \times 0)$ (this takes a few seconds on any reasonable computer).
- Train a *deep residual neural network* to distinguish the real from the random samples.
- For 5 to 7 rounds of Speck encryption, better classifier than DDT after a few minutes of training on GPU (GTX 1080 Ti).
- For seven rounds, a more expensive training scheme gives significant further improvement to network performance.
- Fails for 8 rounds. Staged training saves the day.

Results



Few-Shot Learning

Can networks learn new cryptographic distributions from small data?

Few-Shot Learning

Can networks learn new cryptographic distributions from small data?

Experiment:

- ① Pre-train a network on 3-round Speck with *random* input difference.
- ② Use the penultimate layer as a feature extractor.
- ③ Train a simple linear predictor (Ridge Regression) on 6-round Speck using only 1 to 50 examples.

Few-Shot Learning

Can networks learn new cryptographic distributions from small data?

Experiment:

- 1 Pre-train a network on 3-round Speck with *random* input difference.
- 2 Use the penultimate layer as a feature extractor.
- 3 Train a simple linear predictor (Ridge Regression) on 6-round Speck using only 1 to 50 examples.

Result:

- Accuracy is above random guessing even with **a single training example**.

Few-Shot Learning

Can networks learn new cryptographic distributions from small data?

Experiment:

- 1 Pre-train a network on 3-round Speck with *random* input difference.
- 2 Use the penultimate layer as a feature extractor.
- 3 Train a simple linear predictor (Ridge Regression) on 6-round Speck using only 1 to 50 examples.

Result:

- Accuracy is above random guessing even with **a single training example**.
- Demonstrates robust feature extraction capabilities.

Basic Attack Idea (11-Round Strategy)

- **1. Extend the Neural Distinguisher:**

Basic Attack Idea (11-Round Strategy)

- **1. Extend the Neural Distinguisher:**
 - Start with a trained **7-round** neural distinguisher.

Basic Attack Idea (11-Round Strategy)

- **1. Extend the Neural Distinguisher:**
 - Start with a trained **7-round** neural distinguisher.
 - Prepend a **2-round differential transition** ($\delta \rightarrow 0x0040/0000$) with prob. $\approx 1/64$.

Basic Attack Idea (11-Round Strategy)

- **1. Extend the Neural Distinguisher:**

- Start with a trained **7-round** neural distinguisher.
- Prepend a **2-round differential transition** ($\delta \rightarrow 0x0040/0000$) with prob. $\approx 1/64$.
- **Free Extension:** Add another round at the start by manipulating plaintexts (since no key addition occurs before the first non-linear operation).

Basic Attack Idea (11-Round Strategy)

- **1. Extend the Neural Distinguisher:**

- Start with a trained **7-round** neural distinguisher.
- Prepend a **2-round differential transition** ($\delta \rightarrow 0x0040/0000$) with prob. $\approx 1/64$.
- **Free Extension:** Add another round at the start by manipulating plaintexts (since no key addition occurs before the first non-linear operation).

Basic Attack Idea (11-Round Strategy)

- **1. Extend the Neural Distinguisher:**

- Start with a trained **7-round** neural distinguisher.
- Prepend a **2-round differential transition** ($\delta \rightarrow 0x0040/0000$) with prob. $\approx 1/64$.
- **Free Extension:** Add another round at the start by manipulating plaintexts (since no key addition occurs before the first non-linear operation).

- **2. Signal Boosting (Neutral Bits):**

Basic Attack Idea (11-Round Strategy)

- **1. Extend the Neural Distinguisher:**

- Start with a trained **7-round** neural distinguisher.
- Prepend a **2-round differential transition** ($\delta \rightarrow 0x0040/0000$) with prob. $\approx 1/64$.
- **Free Extension:** Add another round at the start by manipulating plaintexts (since no key addition occurs before the first non-linear operation).

- **2. Signal Boosting (Neutral Bits):**

- The raw signal is weak, so we use k **probabilistic neutral bits**.

Basic Attack Idea (11-Round Strategy)

- **1. Extend the Neural Distinguisher:**

- Start with a trained **7-round** neural distinguisher.
- Prepend a **2-round differential transition** ($\delta \rightarrow 0x0040/0000$) with prob. $\approx 1/64$.
- **Free Extension:** Add another round at the start by manipulating plaintexts (since no key addition occurs before the first non-linear operation).

- **2. Signal Boosting (Neutral Bits):**

- The raw signal is weak, so we use k **probabilistic neutral bits**.
- Create a **plaintext structure** of 2^k pairs that likely pass the initial differential together.

Basic Attack Idea (11-Round Strategy)

- **1. Extend the Neural Distinguisher:**

- Start with a trained **7-round** neural distinguisher.
- Prepend a **2-round differential transition** ($\delta \rightarrow 0x0040/0000$) with prob. $\approx 1/64$.
- **Free Extension:** Add another round at the start by manipulating plaintexts (since no key addition occurs before the first non-linear operation).

- **2. Signal Boosting (Neutral Bits):**

- The raw signal is weak, so we use k **probabilistic neutral bits**.
- Create a **plaintext structure** of 2^k pairs that likely pass the initial differential together.

Basic Attack Idea (11-Round Strategy)

- **1. Extend the Neural Distinguisher:**

- Start with a trained **7-round** neural distinguisher.
- Prepend a **2-round differential transition** ($\delta \rightarrow 0x0040/0000$) with prob. $\approx 1/64$.
- **Free Extension:** Add another round at the start by manipulating plaintexts (since no key addition occurs before the first non-linear operation).

- **2. Signal Boosting (Neutral Bits):**

- The raw signal is weak, so we use k **probabilistic neutral bits**.
- Create a **plaintext structure** of 2^k pairs that likely pass the initial differential together.

Basic Attack Idea (11-Round Strategy)

- **3. Two-Stage Filtering:**

Basic Attack Idea (11-Round Strategy)

- **3. Two-Stage Filtering:**
 - Decrypt the structure using all final subkey guesses.

Basic Attack Idea (11-Round Strategy)

- **3. Two-Stage Filtering:**

- Decrypt the structure using all final subkey guesses.
- **Stage 1:** Check against threshold c_1 using the 7-round distinguisher.

Basic Attack Idea (11-Round Strategy)

- **3. Two-Stage Filtering:**

- Decrypt the structure using all final subkey guesses.
- **Stage 1:** Check against threshold c_1 using the 7-round distinguisher.
- **Stage 2:** If it passes, decrypt one more round and check against threshold c_2 using a **6-round** distinguisher.

The Real Differences Experiment

- **Motivation:** To prove that the neural network learns features *beyond* just the Difference Distribution Table (DDT).

The Real Differences Experiment

- **Motivation:** To prove that the neural network learns features *beyond* just the Difference Distribution Table (DDT).
- **The Setup:** Distinguish between two types of pairs:

The Real Differences Experiment

- **Motivation:** To prove that the neural network learns features *beyond* just the Difference Distribution Table (DDT).
- **The Setup:** Distinguish between two types of pairs:
 - ① **Real Pair:** Standard ciphertext pair (C_0, C_1) from Speck.

The Real Differences Experiment

- **Motivation:** To prove that the neural network learns features *beyond* just the Difference Distribution Table (DDT).
- **The Setup:** Distinguish between two types of pairs:
 - ① **Real Pair:** Standard ciphertext pair (C_0, C_1) from Speck.
 - ② **Blinded Pair:** $(C_0 \oplus K, C_1 \oplus K)$ where K is a random mask.

The Real Differences Experiment

- **Motivation:** To prove that the neural network learns features *beyond* just the Difference Distribution Table (DDT).
- **The Setup:** Distinguish between two types of pairs:
 - ① **Real Pair:** Standard ciphertext pair (C_0, C_1) from Speck.
 - ② **Blinded Pair:** $(C_0 \oplus K, C_1 \oplus K)$ where K is a random mask.
- **The Catch:** Both pairs have the **exact same XOR difference** $(C_0 \oplus C_1)$.

The Real Differences Experiment

- **Motivation:** To prove that the neural network learns features *beyond* just the Difference Distribution Table (DDT).
- **The Setup:** Distinguish between two types of pairs:
 - ① **Real Pair:** Standard ciphertext pair (C_0, C_1) from Speck.
 - ② **Blinded Pair:** $(C_0 \oplus K, C_1 \oplus K)$ where K is a random mask.
- **The Catch:** Both pairs have the **exact same XOR difference** $(C_0 \oplus C_1)$.
 - A standard differential attacker (DDT) sees identical distributions and guesses with 50% accuracy (Failure).

The Real Differences Experiment

- **Motivation:** To prove that the neural network learns features *beyond* just the Difference Distribution Table (DDT).
- **The Setup:** Distinguish between two types of pairs:
 - ① **Real Pair:** Standard ciphertext pair (C_0, C_1) from Speck.
 - ② **Blinded Pair:** $(C_0 \oplus K, C_1 \oplus K)$ where K is a random mask.
- **The Catch:** Both pairs have the **exact same XOR difference** $(C_0 \oplus C_1)$.
 - A standard differential attacker (DDT) sees identical distributions and guesses with 50% accuracy (Failure).
- **The Result:**

The Real Differences Experiment

- **Motivation:** To prove that the neural network learns features *beyond* just the Difference Distribution Table (DDT).
- **The Setup:** Distinguish between two types of pairs:
 - ① **Real Pair:** Standard ciphertext pair (C_0, C_1) from Speck.
 - ② **Blinded Pair:** $(C_0 \oplus K, C_1 \oplus K)$ where K is a random mask.
- **The Catch:** Both pairs have the **exact same XOR difference** $(C_0 \oplus C_1)$.
 - A standard differential attacker (DDT) sees identical distributions and guesses with 50% accuracy (Failure).
- **The Result:**
 - The Neural Network achieves **70.7% accuracy** (for 5 rounds).

The Real Differences Experiment

- **Motivation:** To prove that the neural network learns features *beyond* just the Difference Distribution Table (DDT).
- **The Setup:** Distinguish between two types of pairs:
 - ① **Real Pair:** Standard ciphertext pair (C_0, C_1) from Speck.
 - ② **Blinded Pair:** $(C_0 \oplus K, C_1 \oplus K)$ where K is a random mask.
- **The Catch:** Both pairs have the **exact same XOR difference** $(C_0 \oplus C_1)$.
 - A standard differential attacker (DDT) sees identical distributions and guesses with 50% accuracy (Failure).
- **The Result:**
 - The Neural Network achieves **70.7% accuracy** (for 5 rounds).
 - **Conclusion:** The network is analyzing the *actual values* of the ciphertext, finding patterns that standard differential cryptanalysis ignores.

Why it works : Adaptive Key Search

- **The Classical Flaw:** Standard Differential Cryptanalysis assumes that if a difference ΔC is likely, then *all* pairs (C_0, C_1) summing to ΔC are equally likely.

Why it works : Adaptive Key Search

- **The Classical Flaw:** Standard Differential Cryptanalysis assumes that if a difference ΔC is likely, then *all* pairs (C_0, C_1) summing to ΔC are equally likely.

Why it works : Adaptive Key Search

- **The Classical Flaw:** Standard Differential Cryptanalysis assumes that if a difference ΔC is likely, then *all* pairs (C_0, C_1) summing to ΔC are equally likely.
- **Real Insight:** Checks if the specific ciphertext values *actually* map back to valid states under valid keys.

Why it works : Adaptive Key Search

- **The Classical Flaw:** Standard Differential Cryptanalysis assumes that if a difference ΔC is likely, then *all* pairs (C_0, C_1) summing to ΔC are equally likely.
- **Real Insight:** Checks if the specific ciphertext values *actually* map back to valid states under valid keys.

Why it works : Adaptive Key Search

- **The Classical Flaw:** Standard Differential Cryptanalysis assumes that if a difference ΔC is likely, then *all* pairs (C_0, C_1) summing to ΔC are equally likely.
- **Real Insight:** Checks if the specific ciphertext values *actually* map back to valid states under valid keys.
- **A Concrete Example from the Paper:**

Why it works : Adaptive Key Search

- **The Classical Flaw:** Standard Differential Cryptanalysis assumes that if a difference ΔC is likely, then *all* pairs (C_0, C_1) summing to ΔC are equally likely.
- **Real Insight:** Checks if the specific ciphertext values *actually* map back to valid states under valid keys.
- **A Concrete Example from the Paper:**

The "Imposter" Pair $(0x58e0bc4, 0x85a4ff6c)$

Why it works : Adaptive Key Search

- **The Classical Flaw:** Standard Differential Cryptanalysis assumes that if a difference ΔC is likely, then *all* pairs (C_0, C_1) summing to ΔC are equally likely.
- **Real Insight:** Checks if the specific ciphertext values *actually* map back to valid states under valid keys.
- **A Concrete Example from the Paper:**

The "Imposter" Pair $(0x58e0bc4, 0x85a4ff6c)$

- **DDT View:** The XOR difference is valid and has a *high probability* ($p \approx 2^{-15.3}$). → DDT says "Real"

Why it works : Adaptive Key Search

- **The Classical Flaw:** Standard Differential Cryptanalysis assumes that if a difference ΔC is likely, then *all* pairs (C_0, C_1) summing to ΔC are equally likely.
- **Real Insight:** Checks if the specific ciphertext values *actually* map back to valid states under valid keys.
- **A Concrete Example from the Paper:**

The "Imposter" Pair $(0x58e0bc4, 0x85a4ff6c)$

- **DDT View:** The XOR difference is valid and has a *high probability* ($p \approx 2^{-15.3}$). → DDT says "Real"
- **Deep View:** When decrypting, this specific pair of values matches the input difference with *very low likelihood* ($p \approx 2^{-35.3}$). → NN says "Fake"

- **Conclusion:** The Neural Network learns to reject these "Imposter Pairs" by analyzing the joint distribution of values, which the DDT ignores.

Differential Cryptanalysis vs. Differential Neural Cryptanalysis

Step	Differential Cryptanalysis	Differential Neural Cryptanalysis
Differential Analysis		

Differential Cryptanalysis vs. Differential Neural Cryptanalysis

Step	Differential Cryptanalysis	Differential Neural Cryptanalysis
Differential Analysis	Find differential trail	

Differential Cryptanalysis vs. Differential Neural Cryptanalysis

Step	Differential Cryptanalysis	Differential Neural Cryptanalysis
Differential Analysis	Find differential trail	Select input difference

Differential Cryptanalysis vs. Differential Neural Cryptanalysis

Step	Differential Cryptanalysis	Differential Neural Cryptanalysis
Differential Analysis	Find differential trail	Select input difference
Data Generation		

Differential Cryptanalysis vs. Differential Neural Cryptanalysis

Step	Differential Cryptanalysis	Differential Neural Cryptanalysis
Differential Analysis	Find differential trail	Select input difference
Data Generation	Generate plaintext pairs with input difference ΔP	

Differential Cryptanalysis vs. Differential Neural Cryptanalysis

Step	Differential Cryptanalysis	Differential Neural Cryptanalysis
Differential Analysis	Find differential trail	Select input difference
Data Generation	Generate plaintext pairs with input difference ΔP	Generate large dataset

Differential Cryptanalysis vs. Differential Neural Cryptanalysis

Step	Differential Cryptanalysis	Differential Neural Cryptanalysis
Differential Analysis	Find differential trail	Select input difference
Data Generation	Generate plaintext pairs with input difference ΔP	Generate large dataset
Feature		

Differential Cryptanalysis vs. Differential Neural Cryptanalysis

Step	Differential Cryptanalysis	Differential Neural Cryptanalysis
Differential Analysis	Find differential trail	Select input difference
Data Generation	Generate plaintext pairs with input difference ΔP	Generate large dataset
Feature	Compute and analyze ciphertext differences ΔC .	

Differential Cryptanalysis vs. Differential Neural Cryptanalysis

Step	Differential Cryptanalysis	Differential Neural Cryptanalysis
Differential Analysis	Find differential trail	Select input difference
Data Generation	Generate plaintext pairs with input difference ΔP	Generate large dataset
Feature	Compute and analyze ciphertext differences ΔC .	Prepare ciphertexts or ciphertext differences as neural network inputs.

Differential Cryptanalysis vs. Differential Neural Cryptanalysis

Step	Differential Cryptanalysis	Differential Neural Cryptanalysis
Differential Analysis	Find differential trail	Select input difference
Data Generation	Generate plaintext pairs with input difference ΔP	Generate large dataset
Feature	Compute and analyze ciphertext differences ΔC .	Prepare ciphertexts or ciphertext differences as neural network inputs.
Labels		

Differential Cryptanalysis vs. Differential Neural Cryptanalysis

Step	Differential Cryptanalysis	Differential Neural Cryptanalysis
Differential Analysis	Find differential trail	Select input difference
Data Generation	Generate plaintext pairs with input difference ΔP	Generate large dataset
Feature	Compute and analyze ciphertext differences ΔC .	Prepare ciphertexts or ciphertext differences as neural network inputs.
Labels	Not needed	

Differential Cryptanalysis vs. Differential Neural Cryptanalysis

Step	Differential Cryptanalysis	Differential Neural Cryptanalysis
Differential Analysis	Find differential trail	Select input difference
Data Generation	Generate plaintext pairs with input difference ΔP	Generate large dataset
Feature	Compute and analyze ciphertext differences ΔC .	Prepare ciphertexts or ciphertext differences as neural network inputs.
Labels	Not needed	Assign labels (e.g., cipher-generated = 1, random = 0)

Differential Cryptanalysis vs. Differential Neural Cryptanalysis

Step	Differential Cryptanalysis	Differential Neural Cryptanalysis
Model		

Differential Cryptanalysis vs. Differential Neural Cryptanalysis

Step	Differential Cryptanalysis	Differential Neural Cryptanalysis
Model	Construct Statistical distinguisher	

Differential Cryptanalysis vs. Differential Neural Cryptanalysis

Step	Differential Cryptanalysis	Differential Neural Cryptanalysis
Model	Construct Statistical distinguisher	Train neural network

Differential Cryptanalysis vs. Differential Neural Cryptanalysis

Step	Differential Cryptanalysis	Differential Neural Cryptanalysis
Model	Construct Statistical distinguisher	Train neural network
Evaluation		

Differential Cryptanalysis vs. Differential Neural Cryptanalysis

Step	Differential Cryptanalysis	Differential Neural Cryptanalysis
Model	Construct Statistical distinguisher	Train neural network
Evaluation	Measure the frequency of expected output differences and calculate statistical bias.	

Differential Cryptanalysis vs. Differential Neural Cryptanalysis

Step	Differential Cryptanalysis	Differential Neural Cryptanalysis
Model	Construct Statistical distinguisher	Train neural network
Evaluation	Measure the frequency of expected output differences and calculate statistical bias.	Evaluate the neural network on unseen test data and measure accuracy.

Differential Cryptanalysis vs. Differential Neural Cryptanalysis

Step	Differential Cryptanalysis	Differential Neural Cryptanalysis
Model	Construct Statistical distinguisher	Train neural network
Evaluation	Measure the frequency of expected output differences and calculate statistical bias.	Evaluate the neural network on unseen test data and measure accuracy.
Key Recovery		

Differential Cryptanalysis vs. Differential Neural Cryptanalysis

Step	Differential Cryptanalysis	Differential Neural Cryptanalysis
Model	Construct Statistical distinguisher	Train neural network
Evaluation	Measure the frequency of expected output differences and calculate statistical bias.	Evaluate the neural network on unseen test data and measure accuracy.
Key Recovery	Rank subkeys by counts	

Differential Cryptanalysis vs. Differential Neural Cryptanalysis

Step	Differential Cryptanalysis	Differential Neural Cryptanalysis
Model	Construct Statistical distinguisher	Train neural network
Evaluation	Measure the frequency of expected output differences and calculate statistical bias.	Evaluate the neural network on unseen test data and measure accuracy.
Key Recovery	Rank subkeys by counts	Rank subkeys by neural network scores

Differential Cryptanalysis vs. Differential Neural Cryptanalysis

Step	Differential Cryptanalysis	Differential Neural Cryptanalysis
Performance Metrics		

Differential Cryptanalysis vs. Differential Neural Cryptanalysis

Step	Differential Cryptanalysis	Differential Neural Cryptanalysis
Performance Metrics	Report success rate, data complexity, time complexity, and memory requirements.	

Differential Cryptanalysis vs. Differential Neural Cryptanalysis

Step	Differential Cryptanalysis	Differential Neural Cryptanalysis
Performance Metrics	Report success rate, data complexity, time complexity, and memory requirements.	Report accuracy, loss, key rank, success rate, training time, and computational cost.

Differential Cryptanalysis vs. Differential Neural Cryptanalysis

Step	Differential Cryptanalysis	Differential Neural Cryptanalysis
Performance Metrics	Report success rate, data complexity, time complexity, and memory requirements.	Report accuracy, loss, key rank, success rate, training time, and computational cost.
Result Interpretation		

Differential Cryptanalysis vs. Differential Neural Cryptanalysis

Step	Differential Cryptanalysis	Differential Neural Cryptanalysis
Performance Metrics	Report success rate, data complexity, time complexity, and memory requirements.	Report accuracy, loss, key rank, success rate, training time, and computational cost.
Result Interpretation	Explainable results	

Differential Cryptanalysis vs. Differential Neural Cryptanalysis

Step	Differential Cryptanalysis	Differential Neural Cryptanalysis
Performance Metrics	Report success rate, data complexity, time complexity, and memory requirements.	Report accuracy, loss, key rank, success rate, training time, and computational cost.
Result Interpretation	Explainable results	Results are based on learned patterns and may be less interpretable.



Aron Gohr.

Improving attacks on round-reduced speck32/64 using deep learning.
In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019*, pages 150–179, Cham, 2019. Springer International Publishing.